

En el presente informe se documenta la creación de la entidad UsuarioSupervisor, UsuarioSupervisorId, el repositorio UsuarioSupervisorRepository.

Se vio la necesidad de crear una nueva tabla intermedia donde guarde la relación de un usuario (editor) y un usuario (supervisor).

CREATE TABLE usuarios_supervisores -- tabla de relacion entre usuarios y sus supervisores

```
(
  id_usuario BIGINT,
  id_supervisor BIGINT,
  PRIMARY KEY(id_usuario, id_supervisor),
  FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario),
  FOREIGN KEY (id_supervisor) REFERENCES usuarios(id_usuario)
);
```

Soluciona el problema donde hay un único jefe. En las empresas modernas, un usuario puede reportar a un líder técnico, a un gerente de proyecto y a un director de área simultáneamente. Entonces la tabla intermedia permite que un mismo id_usuario aparezca en múltiples filas con distintos id_supervisor

UsuarioSupervisor

Aca se resalta que es una llave compuesta, por ello se es necesario usar @EmbeddedId, se incrusta lo de UsuarioSupervisorId a la entidad UsuarioSupervisor

```
package com.example.demo.models;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

//modelo para la tabla de relacion de un usuario con su supervisor
@Getter //genera los getters automaticamente con lombok
@Setter //genera los setters automaticamente con lombok
@NoArgsConstructor // Genera el constructor vacío (que pide JPA)
//@AllArgsConstructor // Genera un constructor con todos los atributos
@Entity
@Table(name = ("usuarios_supervisores"))
public class UsuarioSupervisor {
    //llave compuesta
    @EmbeddedId//anotacion para definir que unos campos de esta clase se
    integran a otra tabla de otra entidad (UsuarioSupervisorId)
    private UsuarioSupervisorId id;

    @ManyToOne(fetch = FetchType.LAZY)//carga de datos perezosa, solo lo
    necesario
    @MapsId("idUserio")
    @JoinColumn(name = "id_usuario", nullable = false)
    private Usuario usuario;

    @ManyToOne(fetch = FetchType.LAZY)
```

```

@MapsId("idSupervisor")
@JoinColumn(name = "id_supervisor", nullable = false)
private Usuario supervisor;

public UsuarioSupervisor(Usuario usuario, Usuario supervisor) {
    this.usuario = usuario;
    this.supervisor = supervisor;
    this.id = new UsuarioSupervisorId(
        usuario.getIdUsuario(),
        supervisor.getIdUsuario()
    );
}
}

```

UsuarioSupervisorId

Esta clase no es una entidad en sí, con `@Embeddable` se le dice a JPA/Hibernate que sus campos como (idUsuario e idSupervisor) se van a incrustar' como columnas dentro de otra tabla (UsuarioSupervisor)

CONCEPTOS IMPORTANTES

Serializar es el proceso de convertir un objeto de Java (que vive en la memoria RAM y tiene una estructura compleja) en una secuencia de bytes (un formato plano que se puede enviar por cable o guardar en un disco).

Se utiliza el serializable como prevención por si se necesita escalar el programa a servidores distribuidos y pueda viajar la información por medio de redes. Pero sobre todo para manejar el caché, si se llena la RAM se pasa esos datos al disco duro, si no fuera serializable java lanzará error

serialVersionUID: Es el control de versiones de la estructura de la clase. Garantiza que los objetos guardados anteriormente sigan siendo reconocibles por el sistema incluso si el código sufre pequeñas modificaciones, evitando errores de lectura de datos.

UsuarioSupervisorRepository

Gestión de acceso a los datos de la relacion entre usuario y supervisor

```

package com.example.demo.repositories;
import com.example.demo.models.UsuarioSupervisorId;
import com.example.demo.models.UsuarioSupervisor;
import com.example.demo.models.Usuario;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface UsuarioSupervisorRepository extends
    JpaRepository<UsuarioSupervisor, UsuarioSupervisorId> {
    List<UsuarioSupervisor> findByUsuario_IdUsuario(Long idUsuario); //metodo
de consulta del id del usuario
    List<UsuarioSupervisor> findBySupervisor_IdUsuario(Long
idSupervisor); //metodo de consulta del id del supervisor

```

}